ENGINE ASSEMBLY DEFECT INSPECTOR

온디바이스 AI 기반 엔진 모형 조립 검사 시스템

최종 결과보고서

프로젝트 핵심 데이터 수집·라벨링·모델 학습에서 끝나지 않고, Jetson TensorRT 실행 - PLC Result/Done - LED/컨베이어 동작 - SQLite 이력 조회까지 하나의 검사 흐름으로 연결했다.

평가 제출용 | 자료 기준 2026.10.20 | v1.1

항목	기준
과정	온디바이스 AI 기반 산업용 소프트웨어 개발 1 기
프로젝트	온디바이스 AI 기반 엔진 모형 조립 검사 시스템
팀장	이승훈
평가 근거 소스	GitHub master 기준 코드 + 제출 증빙 문서
운영 Release	engine-integrated-32c484ac6aeb07f1
README 반영 커밋	6b8931aa5c6704ee0b2c6c2a120d3e097b9d9e84

0. 요약 - 평가자가 먼저 확인할 결과

이 프로젝트의 핵심은 단일 객체탐지 모델의 정확도를 보여주는 데 있지 않다. 촬영·라벨·학습·최적화·Jetson 런타임·PLC 제어·DB 이력을 하나의 산업용 검사 흐름으로 연결하고, 각 단계의 결과를 원본 파일과 검사 ID 로 추적 가능하게 만든 것이 주요 성과다.

성과 영역	확인 결과
데이터	고유 원본 216 장. M1·M2 각각 train 144 / val 72. 전체 제출 증빙에 사진·라벨 포함
모델	M1·M2 모두 mAP50 0.995. M1 P 0.99770 / R 1.0, M2 P 0.96041 / R 1.0
Edge 최적화	동일 Jetson PT/TRT 12 회, 4,320 latency 구간. TensorRT 프로세스 RSS 관측 최대 약 49% 감소
실물 통합	정상 4 건 PASS + 오른쪽 파이프 결품 3 건 FAIL = 7/7 계획 판정 일치
PLC·DB	Result/Done/주기 완료 각 7 건, 원본 21 장, 동일 ID 상세·검색 조회 14/14
운영	systemd 기반 단일 통합 서비스, M2→Pose→M1→3 프레임 판정→저장→PLC→Web 구조

핵심 해석 실물 FAIL 3 건은 시험 실패가 아니라 오른쪽 파이프 결품을 의도대로 검출한 결과다. 오조립 분류는 참고 기능으로 남기고 필수 부품 존재/결품을 최종 판정 기준으로 고정했다.

평가영역 대응

평가영역	본 보고서 위치
1 현장 문제·시스템 설계	1~2 장
2 데이터셋·라벨링	3 장
3 Baseline 모델·오류 분석	4 장
4 Jetson 실시간 통합	5~6 장
5 경량화·Benchmark	7 장
6 DB·장치 제어	8 장
7 통합 테스트·현장변수 대응	9 장
8 문서화·시연·발표	10~11 장

1. 현장 문제와 프로젝트 목표

조립형 제품의 외관 검사에서는 필수 부품의 누락을 반복 확인해야 하며, 단순 PASS/FAIL 뿐 아니라 어떤 부품이 문제였는지와 당시 영상을 나중에 다시 확인할 수 있어야 한다. 본 프로젝트는 엔진 모형을 이용해 이 문제를 재현하고, 검사 위치에 도착한 제품을 현장 Edge 장치에서 자동 판정하는 소형 검사 시스템을 구현했다.

구분	정의
주 사용자	검사 설비 운영자. 제품 투입·설비 운전·결과 확인·이력 조회
검사 대상	회색 파이프 2 자리, 배기 부품 1 개, 문양 부품 1 개
입력	PLC 검사 요청·설비 상태 + USB 카메라 새 1280×720 영상
출력	PASS / FAIL / REVIEW / ERROR, 결품 사유, PLC OK/NG, 검사 이력
Edge AI 이유	카메라·추론·판정·저장을 Jetson 에서 수행해 외부 클라우드 응답에 의존하지 않는 현장 검사 흐름 구성

성공 기준

- 정상과 필수 부품 결품을 자동 판정하고, 3 프레임 근거와 reason code 를 남긴다.
- AI 결과가 PLC Result/Done 과 연결되고 다음 공정 동작으로 이어진다.
- 원본 사진·판정·모델/설정·PLC 이벤트를 동일 inspection ID 로 저장·조회한다.
- 학습 모델을 ONNX/TensorRT 로 변환하고 같은 Jetson 에서 최적화 전후를 정량 비교한다.

범위 결정 초기 계획의 오조립·다양한 불량 확장보다 실제 장비에서 안정적인 필수 부품 존재/결품 검사를 우선 완성했다. 현재 오조립 분류는 참고 정보이며 최종 PASS/FAIL 에는 반영하지 않는다.

2. 시스템 구조와 전체 처리 흐름

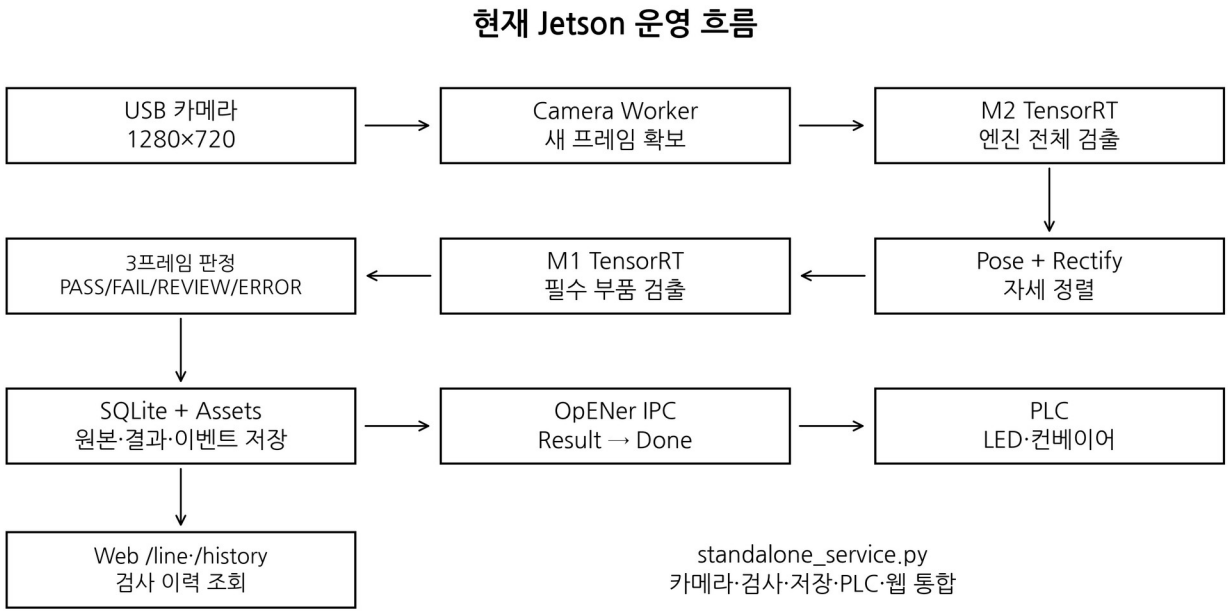


그림 1. PLC·카메라·Jetson·DB·Web 으로 이어지는 현재 운영 구조

구성	역할
PLC / 포토센서	제품 도착과 검사 요청, Result/Done 소비, 컨베이어·LED 제어
USB 카메라	검사 시점의 새 1280×720 원본 프레임 제공
Jetson Orin Nano	M2 엔진 검출, Pose, M1 부품 검출, 3 프레임 판정, 저장·PLC·Web 통합
SQLite / Evidence	검사 원장, 최종판정, 이벤트, 자산 경로·해시 저장
Web /line·/history	현재 검사 결과 표시, 이력 검색·상세·CSV 조회

운영 판정 흐름

PLC 요청이 들어오면 Jetson 은 기존 화면을 재사용하지 않고 새 프레임을 확보한다. M2 가 엔진 전체를 찾고 Pose 가 신뢰 가능한 경우 canonical 좌표로 정렬한 뒤 M1 이 필수 부품을 검출한다. 세 프레임의 자리별 상태를 집계하여 최종 상태를 확정하고, 저장 성공과 동일 요청의 완료 조건을 확인한 뒤 PLC 에 Result 와 Done 을 전달한다.

3. 데이터셋 구축과 라벨링

M1 부품 검출과 M2 엔진 전체 검출은 같은 고유 촬영 원본 216 장을 공유하지만 서로 다른 라벨을 사용한다. 개발 분할은 학습 144 장과 E03 검증 72 장으로 구성했으며, 제출용 증빙에는 두 모델별 전체 사진과 라벨을 포함했다.

모델 / 분할	이미지	라벨 TXT	정답 box	빈 TXT
M1 학습	144	144	384	24
M1 검증	72	72	192	12
M2 학습	144	144	144	0
M2 검증	72	72	72	0

상태	학습	검증	M1 기대 부품
정상	24	12	pipe 2 + exhaust 1 + symbol 1
왼쪽 파이프 결품	24	12	pipe 1 + exhaust 1 + symbol 1
오른쪽 파이프 결품	24	12	pipe 1 + exhaust 1 + symbol 1
배기 결품	24	12	pipe 2 + symbol 1
문양 결품	24	12	pipe 2 + exhaust 1
전체 대상 부품 결품	24	12	M1 box 0 / M2 body box 1

라벨 설계의 핵심 좌·우 파이프는 모두 gray_pipe 클래스다. 실제 어느 쪽이 빠졌는지는 화면 좌우가 아니라 제품 기준 자리와 촬영 상태 기록으로 판단한다. 전체 결품 36 장의 M1 빈 TXT 는 오류가 아니라 “검출 대상 부품 0 개”를 표현한 정상 라벨이다.

4. 모델 학습·평가와 오류 분석

두 모델 모두 YOLO11n 을 기반으로 학습했다. M1 은 필수 부품 3 클래스, M2 는 엔진 전체 1 클래스다. 대표 수치는 학습 중 마지막 epoch 가 아니라 best.pt 를 다시 불러 E03 72 장으로 수행한 개발 검증 결과를 사용했다.

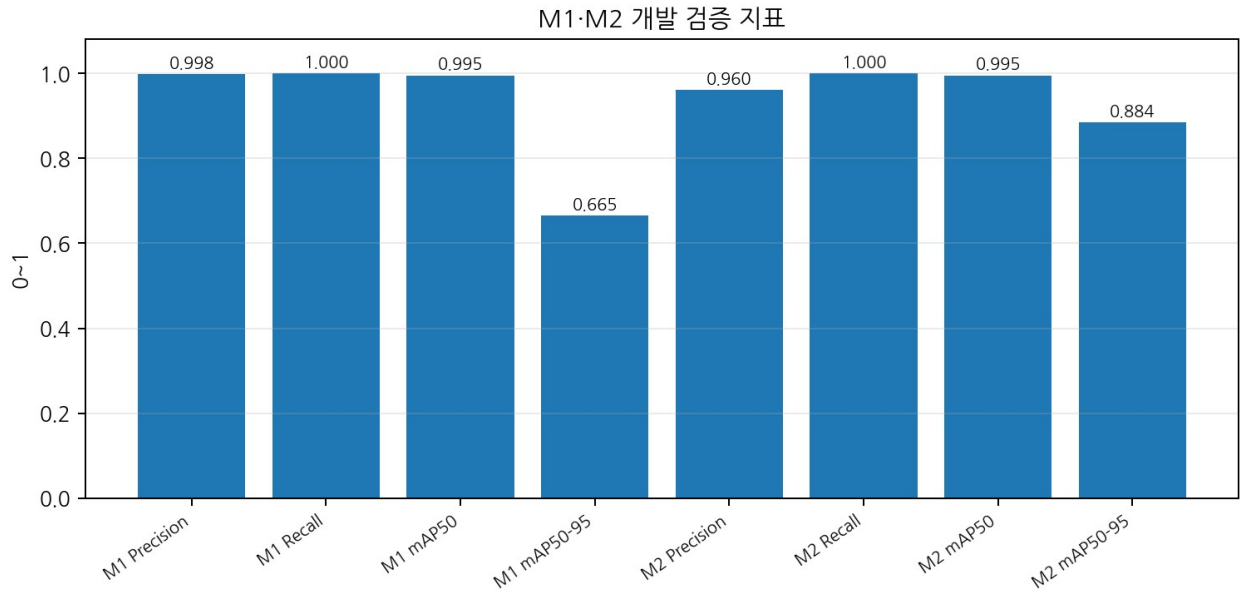


그림 2. M1·M2 개발 검증 주요 지표

모델	실제 학습	Precision	Recall	mAP50	mAP50-95
M1	100 epoch	0.99770	1.00000	0.99500	0.66473
M2	57 epoch (best 37)	0.96041	1.00000	0.99500	0.88427

오류·한계에서 얻은 개선 포인트

- M1 사후 진단(conf 0.25, GT IoU>=0.50)에서는 정답 192 개에 TP 192, FP 0, FN 0 을 기록했다. 그러나 배기 부품의 박스 경계 IoU 가 상대적으로 낮은 사례가 남았다.
- M2 보존 혼동행렬에는 정답 대응 72 개와 배경 쪽 추가 검출 6 개가 기록되어 있어 낮은 confidence 후보와 운영 조건을 구분해 해석했다.
- mAP50 이 높아도 mAP50-95 차이가 존재하므로 단순 “정확도 99.5%”가 아니라 위치 정밀도와 제품 판정을 분리했다.

5. Jetson 실행 코드와 운영 Runtime

기준 운영 Release 는 engine-integrated-32c484ac6aeb07f1 이다. systemd 의 engine-inspector.service 가 단일 통합 서비스를 감독하고, standalone_service.py 가 카메라·검사·PLC·DB·Web 컴포넌트를 하나의 런타임으로 구성한다.

핵심 코드	역할
standalone_service.py	설정 검증, ownership lock, camera/inference/PLC/Web 통합 기동
detector_factory.py	manifest backend 에 따라 TensorRT 또는 PyTorch adapter 선택. silent fallback 없음
tensorrt_preprocess.py	640x640 letterbox, padding114, BGR→RGB, /255
tensorrt_native.py	TensorRT I/O 검증, pinned host, H2D, execute_async_v3, sync, D2H
tensorrt_postprocess.py	confidence, class-aware NMS, 원본 좌표 복원, 진단 metadata
engine_inspector.py	M2 envelope → Pose → rectify → M1 parts → slot decision

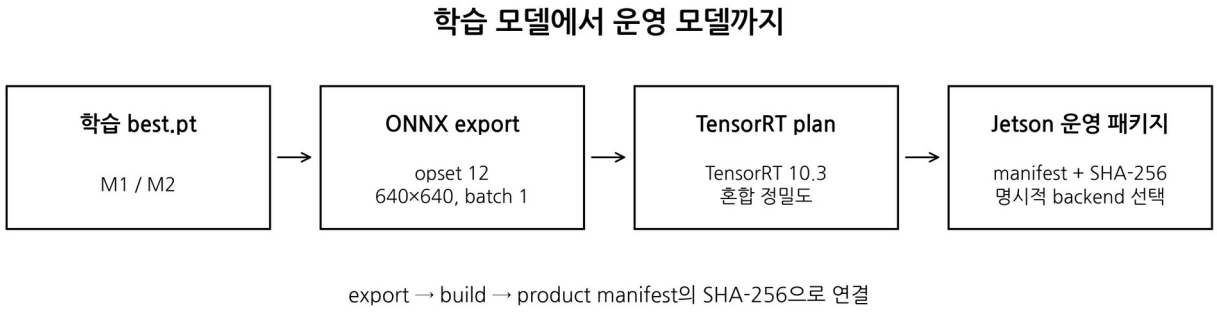


그림 3. 학습 PT 에서 ONNX·TensorRT 운영 모델로 이어지는 변환 계보

6. PT → ONNX → TensorRT 변환과 운영 모델 계보

모델	PT SHA-256 (축약)	ONNX SHA-256 (축약)	운영 TensorRT SHA-256 (축약)
M1	42db1830...bf5259	433b16fd...9c296	743ce54b...bf2d6c
M2	cf117def...0f7755	5cb024bb...ca830	c82f2658...cebf83

해시 연결은 파일명 추정이 아니라 export 기록 → build manifest → 제품 manifest → release / Jetson 관측을 따라 대조했다. M1 과 M2 의 TensorRT plan 은 순수 FP16 으로 단순화하지 않았으며, 각각 일부 FP32 레이어와 float32 I/O 계약을 포함한다.

실행 계약

항목	M1 / M2 공통 원칙
입력	batch1, [1,3,640,640], 고정 shape
전처리	letterbox, padding 114, RGB 변환, /255, contiguous
후처리	class-aware NMS, max_det 300, inverse letterbox
실행	native TensorRT execute_async_v3 + stream synchronize
장치	Jetson Orin Nano, Ubuntu22.04 / L4T36.4.7, CUDA12.6, TensorRT10.3 관측

운영 안전성 manifest 와 I/O 계약이 맞지 않으면 실행 전에 거부하도록 만들었다. TensorRT 가 실패했을 때 다른 backend 로 몰래 전환하지 않으므로, “운영 중인 모델이 무엇인지”를 추적할 수 있다.

7. 동일 Jetson 최적화 전후 Benchmark

같은 Jetson 에서 E03 72 장을 사용해 M1/M2 의 원본 PyTorch FP32 와 운영 TensorRT 를 비교했다. 각 조건을 새 프로세스에서 3 회 실행하고, 예열 50 회 후 72 장을 5 회 순환해 조건당 1,080 개, 총 4,320 개 지연시간을 기록했다.

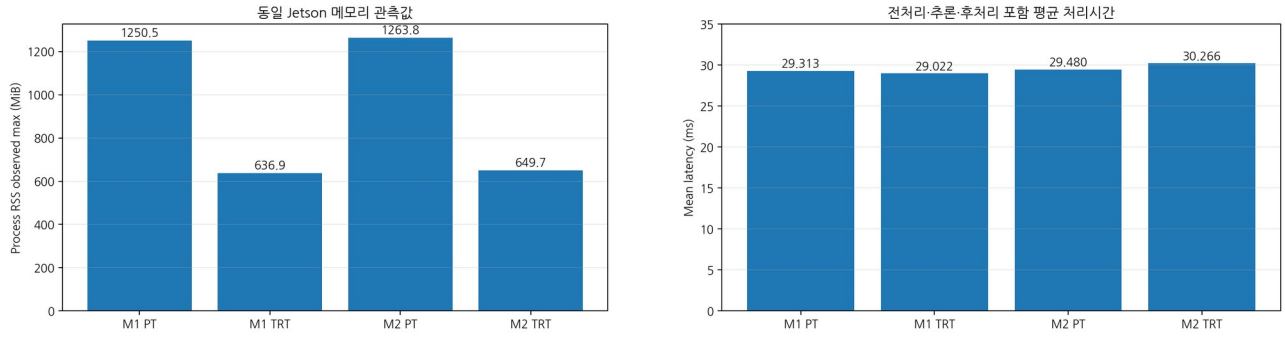


그림 4. 동일 장비 PT/TRT 비교 - 메모리 절감이 가장 분명한 최적화 성과였고, 평균 처리시간은 약 30ms 수준을 유지했다.

항목	M1	M2
평균 지연시간	29.313 → 29.022 ms (-0.99%)	29.480 → 30.266 ms (+2.67%)
처리량	34.06 → 34.41 image/s	33.87 → 33.00 image/s
RSS 관측 최대	1,250.5 → 636.9 MiB (-49.1%)	1,263.8 → 649.7 MiB (-48.6%)
고정조건 TP/FP/FN	192/0/0 → 192/0/0	72/0/0 → 72/0/0
mAP50-95	0.639911 → 0.640873	0.836717 → 0.836629

Benchmark 결론 TensorRT 적용 효과를 “몇 배 가속”으로 과장하지 않았다. 이번 구성의 대표 성과는 프로세스 RSS 약 49% 절감이며, E03 고정 조건의 검출 결과는 PT 와 TRT 가 근접했다.

8. 실제 라인 통합: AI → PLC → 장치 → DB

2026-10-18 시험에서 정상 모형과 오른쪽 파이프 결품 모형을 번갈아 투입했다. 계획 10 회 중 사용자 종료 의사에 따라 실제 7 회가 저장됐고, 동일 boot·실행본·카메라 Worker·PLC 연결에서 수행됐다.

순번	실제 상태	최종 판정	사유	PLC Result
1	정상	PASS	ALL_REQUIRED_PARTS_PRESENT_3_FRAMES	0 / OK
2	오른쪽 파이프 결품	FAIL	RIGHT_PIPE_MISSING	1 / NG
3	정상	PASS	ALL_REQUIRED_PARTS_PRESENT_3_FRAMES	0 / OK
4	오른쪽 파이프 결품	FAIL	RIGHT_PIPE_MISSING	1 / NG
5	정상	PASS	ALL_REQUIRED_PARTS_PRESENT_3_FRAMES	0 / OK
6	오른쪽 파이프 결품	FAIL	RIGHT_PIPE_MISSING	1 / NG
7	정상	PASS	ALL_REQUIRED_PARTS_PRESENT_3_FRAMES	0 / OK



정상 투입 사례



오른쪽 파이프 결품 사례

그림 5. 실제 7 회 통합시험에서 회수한 원본 프레임 예시. 검사별 원본 3 장을 저장했으며 총 21 장의 실파일을 원장 해시와 대조했다.

9. 검사 이력과 추적성

SQLite 는 단순 PASS/FAIL 저장소가 아니라 검사 트랜잭션을 추적하는 원장으로 사용했다. 사진은 파일시스템에 보존하고 DB 에는 inspection ID, 판정, 이벤트, 파일 경로·크기·SHA-256 과 구조화된 결과를 저장한다.

연결 키	사용 목적
inspection_id	검사 원장·결품·사진·최종판정·이벤트 공통 식별
cell_id + plc_session_id + cycle_id	공정 cycle 과 검사 원장 연결
event_id	events 와 outbox 전송 상태 연결
상대 경로 + SHA-256	DB 기록과 실제 사진 파일 동일성 대조

현재 스키마 검토에서는 10 개 테이블·67 개 열을 확인했고, 통합시험 7 건에 대해 상세·검색 조회 14 회가 기존 원장과 일치했다.



그림 6. Jetson 웹 이력 화면 - 저장된 검사 ID, 결품 사유와 판정 이미지를 조회

추적성의 의미 평가자는 “화면에 FAIL 이 떴다”만 보는 것이 아니라, 같은 inspection ID 로 실제 원본·결품 사유·PLC 이벤트·DB 저장·조회 결과를 따라갈 수 있다.

10. 계획 대비 달성 결과와 역할

계획 목표	현재 결과	판정
카메라 기반 부품 검사	M1 부품 + M2 엔진 검출, 새 원본 3 프레임 기반 판정	달성
Jetson 실시간 실행	운영 release 32c4, native TensorRT, systemd 통합 서비스	달성
PLC·장치 연계	실물 7 건 Result/Done 완료, LED·컨베이어 동작 운영자 확인	달성
DB 저장·조회	7 건 COMPLETE, 원본 21·표시 7, 상세/검색 14/14 일치	달성
경량화·성능 비교	동일 Jetson 12 회·4,320 latency, RSS 약 49% 감소	달성
오조립 최종 판정	참고 분류 모델과 표시 기능은 있으나 최종 판정 미반영	범위 조정
푸셔 자동 분류	현재 PLC 는 푸셔 없는 구성	제외

계획 당시 역할 분담

담당	계획서 기준 역할
이승훈 / Vision·AI	검사 대상·라벨 기준, 모델 학습·평가, 검사 규칙, ONNX/TensorRT 최적화
이규헌 / Edge·Automation	Jetson·카메라, PLC 통신, 검사 요청·결과, 컨베이어 이송·복구
주현규 / Data·QA·UI	데이터 검수·분할, DB 저장, 검사 화면·이력, 시험·증거 정리

위 역할은 2026-09-11 프로젝트 계획서에 기록된 분담이며, 실제 개인별 코드 기여량을 재산정한 표는 아니다.

11. 한계와 개선 방향

현재 결과는 구현 완성도를 보여주지만 모든 산업 환경을 대표하는 인증 시험은 아니다. 평가자가 성과의 범위를 정확히 해석할 수 있도록 확인된 한계와 다음 검증을 분리했다.

현재 한계	영향	다음 보완
실물 반복시험이 정상·오른쪽 파이프 결품 7 건 중심	다른 결품·복합 불량 일반화 범위 제한	배기·문양·좌측 파이프·복합 결품 실물 반복
조명·거리·각도 체계적 현장시험 미완료	환경 변화 강건성 정량화 부족	변수 하나씩 바꾼 시험표와 실패 사례 저장
장시간 연속운전 미인증	장기 안정성·USB 복구 빈도 미정량	30~60 분 반복 운전과 복구 횟수 기록
검사별 실제 장치 동작 영상 미확보	LED·이송을 제 3 자가 독립적으로 재확인하기 어려움	최신 운영본 정상→결품→정상 5 분 시연영상
E04 독립 최종시험 완료 근거 없음	개발 검증과 독립 최종 성능 구분 필요	보호된 holdout 을 최종 한 번 평가하고 결과 동결

개선 방향의 우선순위 마감 전 가장 가치가 큰 보완은 최신 운영본의 실제 시연영상이다. 모델을 다시 튜닝하기보다 정상→결품→정상 한 흐름에서 AI 판정, LED/이송, 동일 ID 이력 조회를 영상으로 연결하는 것이 평가표 8 번에 직접 대응한다.

12. 결론

본 프로젝트는 엔진 모형을 대상으로 데이터 구축, 객체탐지 학습, TensorRT 변환, Jetson 실행, PLC 통신, 현장 장치 반응, SQLite 저장과 웹 조회를 하나의 검사 시스템으로 연결했다. 특히 모델 성능을 제시하는 데서 멈추지 않고 실제 장비에서 7 건의 검사 흐름을 완료하고, 동일 Jetson 에서 원본 PyTorch 와 TensorRT 를 정량 비교했다.

최종 성과	대표 근거
AI 모델	M1/M2 mAP50 0.995, 실제 오류·경계 사례 분석
Edge 실행	학습 PT→ONNX→TRT→운영 release 해시 계보
최적화	TensorRT 프로세스 RSS 관측 최대 약 49% 감소
통합 운전	정상 4 PASS + 결품 3 FAIL, 저장·PLC 완료 각 7 건
추적성	원본 21 장, 표시 7 장, 동일 ID 조회 14/14, SQLite/Web 구현

프로젝트의 가치 단일 YOLO 데모가 아니라 “데이터 → 모델 → Edge → PLC → DB → 시연”의 전 과정을 실제 장비와 근거 파일로 연결한 산업용 소프트웨어 프로젝트라는 점이 핵심 강점이다.

부록 A. 제출 산출물과 근거 연결

ID	산출물	역할
R01	01_엔진모형_문제정의서_v1.0	현장 문제·사용자·입출력·성공 기준
R02	02_엔진모형_데이터셋_설명서_v1.2 + 전체 데이터 ZIP	수집·라벨·분할·전체 원본/라벨
R03	03_엔진모형_모델_평가서_v1.1 + 모델 평가 증빙 ZIP	학습·P/R/mAP·오류 사례
R04	04_엔진모형_최적화전후_Benchmark_보고서_v1.1	동일 Jetson PT/TRT 비교
R05	05_엔진모형_통합테스트_결과서_v1.0	실물 7 건 AI→PLC→DB
R06	06_엔진모형_DB_구조와_조회방식_v1.0	SQLite/API/조회 구조
R07	07_엔진모형_Jetson_실행_ONNX_TensorRT_절차서_v1.0	실행 코드·변환·설치·운영
R08	08_엔진모형_프로젝트_기획안_v1.0	계획 당시 역할과 목표 대비 결과
R09	GitHub README / master	현재 구조·성과·실행·제한의 공개 안내

부록 B. 기준 버전

대상	기준
운영 Release	engine-integrated-32c484ac6aeb07f1
통합 실물시험 실행본	engine-integrated-4c4b935f8ab0404a
평가 근거 코드 기준	GitHub master 의 운영/학습 소스
README 최신화 커밋	6b8931aa5c6704ee0b2c6c2a120d3e097b9d9e84
M1 TRT	743ce54b...bf2d6c
M2 TRT	c82f2658...cebf83

주의: README 최신화 커밋은 문서 업데이트이며 운영 Runtime 바이너리를 다시 배포한 기록이 아니다. 실물 7 회 시험은 4c4b 에서 수행됐고, 후속 32c4 는 UI 변경 범위를 별도 검증했다.